

ForeSight: Small Calibrated Decision Models as Opponent Forecasters for Lookahead Game Play

Saiakhil Chilaka

Cornell University
314 West Ave, Ithaca, NY 14853
(984) 260-2526
sc3484@cornell.edu

Abstract

To choose a good move, an agent has to guess how its opponent will respond. Game engines make that guess with hand-written rules and an exact simulator, which settings like negotiation do not have. LLM agents make it with large generative models, which are slow and poorly calibrated. We propose ForeSight, which fine-tunes a small, non-generative “System One” decision model to give a calibrated probability for each opponent action, then uses those probabilities to weight every branch of a lookahead search. In Pokémon Showdown, a 421M-parameter forecaster trained for 35 minutes on a laptop, using only games between weak bots, predicts the opponent’s next action 72–80% of the time (43% zero-shot, 49% for a damage-based rule). It reaches 75% on an opponent it never saw in training, at 36–84 ms per forecast. With no LLM, ForeSight wins 56.5% of 200 games against the Abyssal benchmark bot. That matches the LLM agent PokéLLMon (56%) but trails PokéChamp (64–70%), and searching one extra turn ahead adds 13–20 points over one-step play. Every part is still minimal, so small calibrated forecasters could bring engine-style lookahead to settings with no engine.

Code — <https://github.com/akhilc08/aaai-27>

Introduction

Good decisions, whether a move in a game or a step in negotiations, depend on predicting how the other side will react. Classical game engines do this with search over an exact simulator and a hand-written evaluator built for the engine. The Pokémon engine Foul Play, for example, pairs tree search with a hand-tuned position score (Mariglia 2026a). That works when the rules are fully known but it breaks down in negotiation and less deterministic states, which has no simulator and no obvious hand-written opponent model.

LLM agents go the other way. In Pokémon Showdown, PokéLLMon asks an LLM to pick each move (Hu, Huang, and Liu 2024), and PokéChamp uses one to propose actions, model the opponent and score positions inside a minimax search (Karten, Nguyen, and Jin 2025). LLMs know a lot about the game, but their stated probabilities are poorly calibrated and each call takes seconds. PokéChamp traces about a third of its losses to running out of turn time.

Copyright © 2027, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

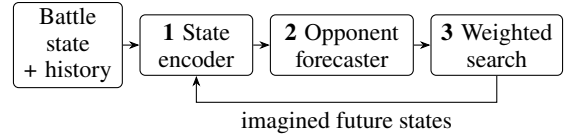


Figure 1: ForeSight. The search weights every imagined future by the forecaster’s opponent probabilities and by chance, then plays the best expected move.

ForeSight splits the problem in two: a model that forecasts and a search that decides. The forecaster is a small System One decision model (TypeSafe AI 2026; Convai Innovations 2026). It reads a state and a list of typed options and returns a calibrated probability for each, in one forward pass and without generating text. A probability-weighted search then turns those forecasts into a move (Figure 1). We test ForeSight in Pokémon Showdown.

ForeSight

1. State encoder. Each position is written out as short precomputed facts: units left, HP, who moves first, estimated damage and knock-out chances for each attack, type matchups, and both sides’ last few actions. We avoid raw text because in a pilot it hid even a decisive lead ($P(\text{win})$ 0.66 vs. 0.87 with facts). The candidate opponent actions are its revealed moves, one placeholder attack per type for unrevealed ones, and switches to Pokémon we have seen.

2. Opponent forecaster. We fine-tune Laya (Convai Innovations 2026), an open 421M-parameter System One model, on 10k opponent decisions sampled from 142k logged in local games between simple rule-based and random bots. The data is weak on purpose. We train the top four encoder layers and the decision head (75M parameters) with Laya’s proper-scoring-rule loss, then fit a temperature on validation data. Training takes 35 minutes on an Apple M5 Pro MacBook.

3. Probability-weighted search. ForeSight runs a depth-2 expectimax. For each of our actions, it branches over the forecaster’s distribution of opponent replies and over chance events (accuracy, damage rolls near a knock-out, secondary effects, speed ties), keeping 90% of the probability mass at each node. Leaves are scored by the HP difference between the two sides, another deliberately crude choice, and we play

Forecaster	Evaluated on	Acc.	Log-loss
Damage-based rule	Gen 9 bots	48.5%	1.24
Laya, zero-shot	Gen 9 bots	43.4%	1.15
ForeSight (Gen 9)	Gen 9 bots	71.8%	0.70
ForeSight (Gen 8)	Gen 8 bots	79.6%	0.54
ForeSight (Gen 9)	Abyssal, unseen	73.0%	0.62
ForeSight (Gen 8)	Abyssal, unseen	75.4%	0.58

Table 1: Next-action forecasting on held-out turns ($n=3,754-4,000$ bot turns; 1,963 Abyssal turns).

Agent	Decision model	Win rate
PokéLLMon [†]	GPT-4o	56%
PokéChamp [†]	Llama-3.1-8B	64%
PokéChamp [†]	GPT-4o	70%
ForeSight (Gen 9)	Laya 421M	56.5%
ForeSight (Gen 8)	Laya 421M	56.5%

Table 2: Win rate against Abyssal. [†]Published by Karten, Nguyen, and Jin (2025). ForeSight: $n=200$ games each, 95% CI 50–63%.

the action with the highest expected value.

Evaluation

Experimental Setup. We test forecasters trained on Gen 9 and Gen 8 games on held-out battles, and compare them with a rule that always predicts the opponent’s strongest attack. For play, we follow PokéChamp’s protocol: Gen 8 random battles without Dynamax against Abyssal, a rule-based bot that never appears in our training data, on a local Showdown server with a 15 s turn limit.

Forecasting. Fine-tuning turns a weak zero-shot model into an accurate, well-calibrated forecaster (calibration error 0.02; Table 1). It also transfers: on Abyssal, which it never saw in training, it predicts 73–75% of actions.

Play. Without an LLM, ForeSight plays about as well as PokéLLMon with a far smaller model, though it trails PokéChamp (Table 2). The Gen 8 forecaster predicts better but wins no more often. Since the forecasts on Abyssal are already accurate, we think the gap to PokéChamp comes mostly from our shallow search and simple leaf.

Speed and depth. One forecast takes 36–84 ms depending on batch size, and a turn takes 2.7–4.1 s on average, so the 15 s limit leaves room for 10–25 $\times$ more search than we use. Lookahead clearly matters. Against Abyssal, searching a second turn ahead adds 13–20 points for every opponent model we tried (42% to 58% with the damage-based forecaster, $n=1000$ each), and our depth-1 result is in line with PokéChamp’s one-step baseline (44%). Outside games, in the text household benchmark ALFWorld (Shridhar et al. 2021), letting a System One model pick among an LLM’s top three actions by predicted outcome raised success from 34.6% to 48.4% ($p<0.001$).

Conclusion and Future Work

ForeSight uses a small, fine-tuned, calibrated decision model as the opponent forecaster inside a lookahead search. Even this bare-bones version matches an LLM agent built on a much larger model. Zero-shot System One models (Laya, and Jev in our pilots) forecast poorly, so fine-tuning is essential.

Headroom. Every component is simple and can improve.

(1) *Deeper search*: we use less than a tenth of the turn budget. (2) *Better data*: trained only on weak bot games, the forecaster already predicts an unseen opponent 75% of the time, and replays of high-rated human games (Grigsby et al. 2025) should push that higher. (3) *A learned leaf*: scoring by HP alone means a setup move like Swords Dance only counts if its extra damage lands within two turns; a learned win-probability leaf could value boosts, status and hazards. (4) *Full fine-tuning and exact rules*: we train only 18% of the parameters, and an exact engine (Mariglia 2026b) with sampled hidden teams would remove known simulator errors. Because the Abyssal forecasts are already accurate, we expect (1), (3) and (4) to do the most to narrow the gap to PokéChamp. Finally, since ForeSight needs no hand-written opponent model, we plan to apply it to negotiation (Lewis et al. 2017), with an LLM proposing the candidate next states.

Acknowledgments

Claude Code (Anthropic) assisted with experiment code, evaluation runs and editing; the author directed the work and is responsible for all content.

References

- Convai Innovations. 2026. Laya: A System 1 Decision Engine. <https://github.com/NandhaKishorM/laya>. Accessed: 2026-09-27.
- Grigsby, J.; Xie, Y.; Sasek, J.; Zheng, S.; and Zhu, Y. 2025. Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers. ArXiv:2504.04395.
- Hu, S.; Huang, T.; and Liu, L. 2024. PokéLLMon: A Human-Parity Agent for Pokémon Battles with Large Language Models. ArXiv:2402.01118.
- Karten, S.; Nguyen, A. L.; and Jin, C. 2025. PokéChamp: an Expert-level Minimax Language Agent. In *International Conference on Machine Learning*.
- Lewis, M.; Yarats, D.; Dauphin, Y.; Parikh, D.; and Batra, D. 2017. Deal or No Deal? End-to-End Learning of Negotiation Dialogues. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2443–2453.
- Mariglia, P. 2026a. Foul Play: A Pokémon Showdown Battle Bot. <https://github.com/pmariglia/foul-play>. Accessed: 2026-09-27.
- Mariglia, P. 2026b. poke-engine: A Pokémon Battle Engine for Search. <https://github.com/pmariglia/poke-engine>. Accessed: 2026-09-27.
- Shridhar, M.; Yuan, X.; Côté, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations*.

TypeSafe AI. 2026. Introducing System One Models and Jev. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. Accessed: 2026-09-27.